بسمه تعالی



تشخیص نفوذ
درس اول : مرور مفاهیم مرتبط با نفوذ

آغاز با مرور یک رخداد

- شرکت مرسک یکی از بزرگترین شرکت های حمل و نقل دریایی جهان است. در سال ۲۰۱۷ این شرکت ۷۶ بندر در سراسر جهان و ۸۰۰ کشتی دریایی را مدیریت می‌کرد که تقریباً یک پنجم ظرفیت حمل و نقل کل سیاره را تشکیل می‌داد.
- در ژوئن ۲۰۱۷ کارمندان به کار روزانه خود مشغول بودند و مانند اکثر شرکت‌ها، خود را هدف بالقوه یک حمله سایبری هدفمند نمی‌دید، و مدیران ریسک آن دامنه تاثیرات بالقوه چنین حمله ای را حدس نمی زنند.



- این درحالی بود که در جریان جنگ روسیه و اوکراین، روسیه حملاتی راعلیه اوکراین آغاز کرد که از کسب‌وکارهای کوچک خانوادگی اوکراینی گرفته تا غول‌های تجاری بین‌المللی چند میلیارد دلاری، فلج کرد.

PETYA OR NOTPETYA?

- **داستان Petya** (پتیا – **۲۰۱۶**) : یک **باج‌افزار تجاری** (Ransomware-as-a-Service) با هدف صرفاً مالی و اخاذی.
- هکرهای جنایی سایبری (احتمالاً روسی/اروپای شرقی به نام مستعار Janus Secrets).
- برخلاف باج‌افزارهای عادی که فایل‌ها (مثل عکس و ورد) را تک‌تک قفل می‌کردند، پتیا **(MBR)** و جدول فایل‌های هارد **(MFT)** را رمزگذاری می‌کرد؛ یعنی سیستم ری‌استارت می‌شد، صفحه آبی/قرمز معروفی با عکس یک جمجمه بالا می‌آمد و ویندوز اصلاً بالا نمی‌آمد!
- قربانی در صورت پرداخت ۳۰۰ دلار بیت‌کوین، یک کلید رمزگشایی واقعی دریافت می‌کرد و هارد دیسک قابل بازیابی بود.



- وقتی حمله در ژوئن ۲۰۱۷ اتفاق افتاد، رسانه‌ها و شرکت‌های آنتی‌ویروس در ساعات اولیه گفتند: نسخه جدید پتیا پخش شده است!
- اما شرکت امنیتی **کاسپرسکی (Kaspersky Labs)** کدهای آن را تحلیل کرد و متوجه شد که این بدافزار با وجود شباهت ظاهری، از پایه معماری متفاوتی دارد و یک باج‌افزار نیست، بلکه یک **Wiper** (**پاک‌کننده اطلاعات**) است. به همین دلیل آن‌ها برای جلوگیری از اشتباه گرفتن آن با باج‌افزار اصلی، روی آن نام **NotPetya** (یعنی: این پتیا نیست!) را گذاشتند.

زمان بندی قابل تامل نفوذ در مرسک

- **۲۰۱۶ – هشدارهای امنیتی درون‌سازمانی:**
  - مدیران ارشد سیستم شرکت کشتیرانی مرسک هشدار دادند که شبکه شرکت با بیش از ۸۰٬۰۰۰ کامپیوتر در برابر حملات سایبری به شدت آسیب‌پذیر است.
  - وجود سرورهای ویندوز ۲۰۰۰ و سیستم‌های ویندوز XP که ارتقا و تعویض آن‌ها به تعویق افتاده بود.
  - مدیریت، بودجه ارتقا را تأیید کرده بود، اما ادمین‌های سیستم انگیزه‌ای برای اجرای ارتقا نداشتند (زیرا سیستم پاداش‌دهی بر اساس پایداری/آپ‌تایم سیستم‌ها تعریف شده بود، نه امنیت).
- **مارس ۲۰۱۷ – انتشار وصله امنیتی :**
  - مایکروسافت یک وصله امنیتی اضطراری برای رفع آسیب‌پذیری پروتکل SMBv۱ (موسوم به EternalBlue) منتشر کرد (که بعداً NotPetya برای گسترش خود در شبکه از آن استفاده کرد).
- **۲۷ ژوئن ۲۰۱۷ – حمله NotPetya (تخریب گسترده):**
  - بدافزار از طریق به‌روزرسانی آلوده نرم‌افزار مالیاتی اوکراینی (M.E.Doc) در یکی از سیستم‌های شعبه اوکراین مرسک نفوذ کرد و در کمتر از چند دقیقه در کل شبکه جهانی پخش شد:
  - **از دست رفتن کامل زیرساخت IT:** بدافزار NotPetya در ظاهر باج‌افزار ولی در واقع یک بدافزار تخریب‌کننده (Wiper) بود که کل سیستم‌ها را غیرقابل بازگشت می‌کرد.
  - **پاک شدن اکتیو دایرکتوری:** تمام ۱۴۰ تا ۱۵۰ دامین کنترلر شرکت که وظیفه احراز هویت مرکزی، کاربران و دسترسی‌ها را بر عهده داشتند، پاک و از دسترس خارج شدند.
  - **توقف عملیات و بازگشت به روش‌های دستی:** بنادر و پایانه‌های مرسک مجبور به مدیریت دستی بارگیری و تخلیه کشتی‌ها با کاغذ و قلم شدند.
  - **نجات معجزه‌آسا از طریق غنا:** تنها ۱ دامین کنترلر در شعبه غنا به دلیل قطعی برق محلی در زمان حمله خاموش و آفلاین بود؛ متخصصان این هارد را به مقر اصلی در انگلستان منتقل کرده و ساختار کل شبکه را از روی آن بازسازی کردند.
  - **خسارت مالی و عملیاتی:** حدود ۱۰ روز اختلال شدید در عملیات تجاری و برآورد ضرر مالی **۲۵۰ تا ۳۰۰ میلیون دلار**.
  - *(نکته آماری: طبق گزارش‌ها، حدود ٪۶۰ از کسب‌وکارهای کوچک و متوسط پس از یک حمله سایبری فاجعه‌بار، ظرف ۶ ماه ورشکست می‌شوند).*
- **ژوئیه ۲۰۱۷ – بازسازی زیرساخت:**
  - در یک عملیات بازسازی اضطراری در طول چند هفته، مرسک اقدام به راه‌اندازی و تعویض حدود **۴٬۰۰۰ سرور جدید، ۴۵٬۰۰۰ رایانه شخصی جدید** و نصب مجدد بیش از **۲٬۵۰۰ برنامه کاربردی** کرد

Nopetyia

- حمله روسی Nopetya با دو ابزاری که انتظار نمی رفت:
- Blueeternal : این یک آسیب پذیری در (پروتکل Server Message Block version 1) سیستم عامل ویندوز بود که NSA پنج سال پیش از آن تاریخ آن را یافته بود اما تصمیم گرفته بود آن را افشا نکند و خود از آن استفاده کند. در جریان هک NSA در سال ۲۰۱۷ این آسیب پذیری لو رفت و بر مبنای آن کرمی توسعه داده شده بود که با سرعت ده هزار ماشین در ساعت ماشین های مختلف را آلوده می کرد . این کرم WannaCry نام داشت.
- Mimitaks هم ابزاری از یک محقق فرانسوی بود که آن را به عنوان اثبات مفهومی توسعه داده بود تا نشان دهد که رمزهای عبور ویندوز را می‌توان از حافظه سیستم بازیابی کرد. بعد از دسترسی به سیستم، مهاجم می‌توانست به تمام دستگاه‌های دیگر در همان شبکه متصل شود و از طریق امتیازات خود به آنها دسترسی دهد

عکس العمل مرسک

- شرکت با سیستم هایی مواجه شد که در قبال پول قابل بازیابی بودند، اما بعدا مشخص شد اطلاعات از بین رفته و حمله باج افزار و قابل بازیابی نبوده است.
- نخستین اقدام شما در چنین شرایطی ؟

عکس العمل مرسک

- شرکت با سیستم هایی مواجه شد که در قبال پول قابل بازیابی بودند، اما بعدا مشخص شد اطلاعات از بین رفته و حمله باج افزار و قابل بازیابی نبوده است.
- نخستین اقدام شما در چنین شرایطی ؟
- قطع کارگزارها (سرور) از شبکه برای پیشگیری از انتشار بیشتر
- فراخوان تیم مشاور امنیتی دیلویت برای حضور در مرکز و تلاش برای بازیابی
  - چه اتفاقی برای سرورهای بک آپ افتاده است؟
  - کدام سرور بک با آپ احتمال بیشتری سالم بود؟

تشخیص (وقوع) نفوذ

- چه زمانی می توانیم ادعا کنیم در سیستم ما نفوذ اتفاق نیفتاده است ؟
  - تا وقتی یک بازیگر ثانویه ادعای نفوذ نکرده است؟
  - تا وقتی سیستم به همان روال قبلی کار می کند ؟
  - وقتی یک سیستم نو خریداری کردیم که مطمانیم هنوز به آن حمله ای نشده است؟

تشخیص (وقوع) نفوذ

!چه زمانی می توانیم ادعا کنیم در سیستم ما نفوذ اتفاق نیفتاده است ؟

✓چه زمانی احتمال نفوذ را پایین تر آورده ایم ؟

! تا وقتی یک بازیگر ثانویه ادعای نفوذ نکرده است؟

! تا وقتی سیستم به همان روال قبلی کار می کند ؟

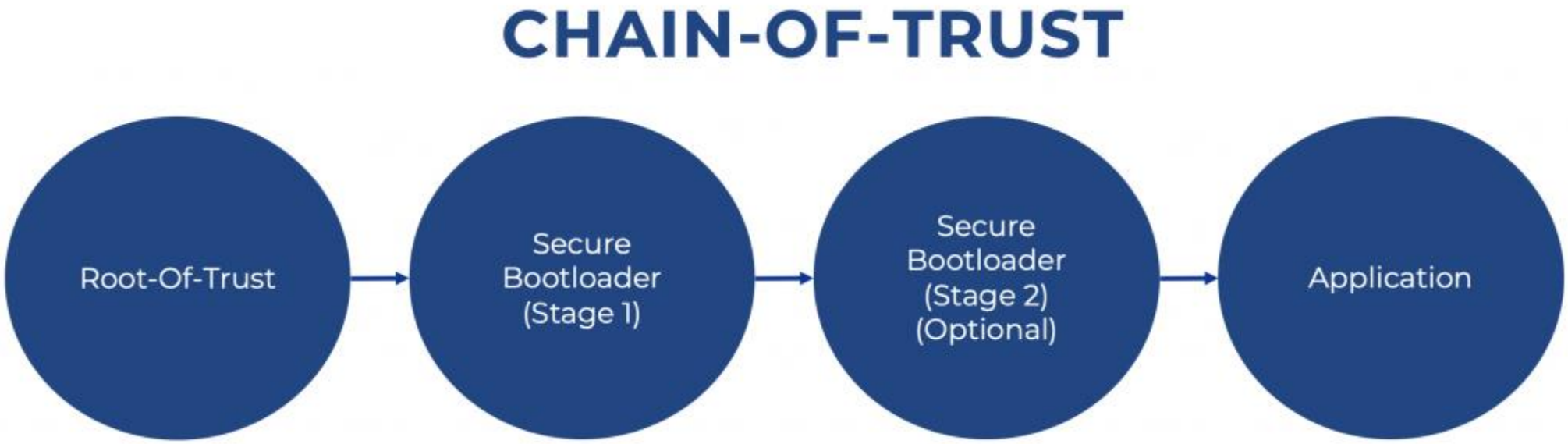
! وقتی یک سیستم نو خریداری کردیم که مطمانیم هنوز به آن حمله ای نشده است؟

✓ تا وقتی ریسک ناشی از زنجیره تامین نرم افزاری و سخت افزاری سیستم را در مرتبه قابل قبولی ارزیابی می کنیم.

✓وقتی اساسا سعی داریم بجای اعتماد، رویکرد بی اعتماد (zero trust) به کار بردیم ؟

✓ وقتی سیستم طوری طراحی شده که ریشه و زنجیره اعتماد در آن تعبیه شده است ؟

ریشه اعتماد به سیستم (RoT)



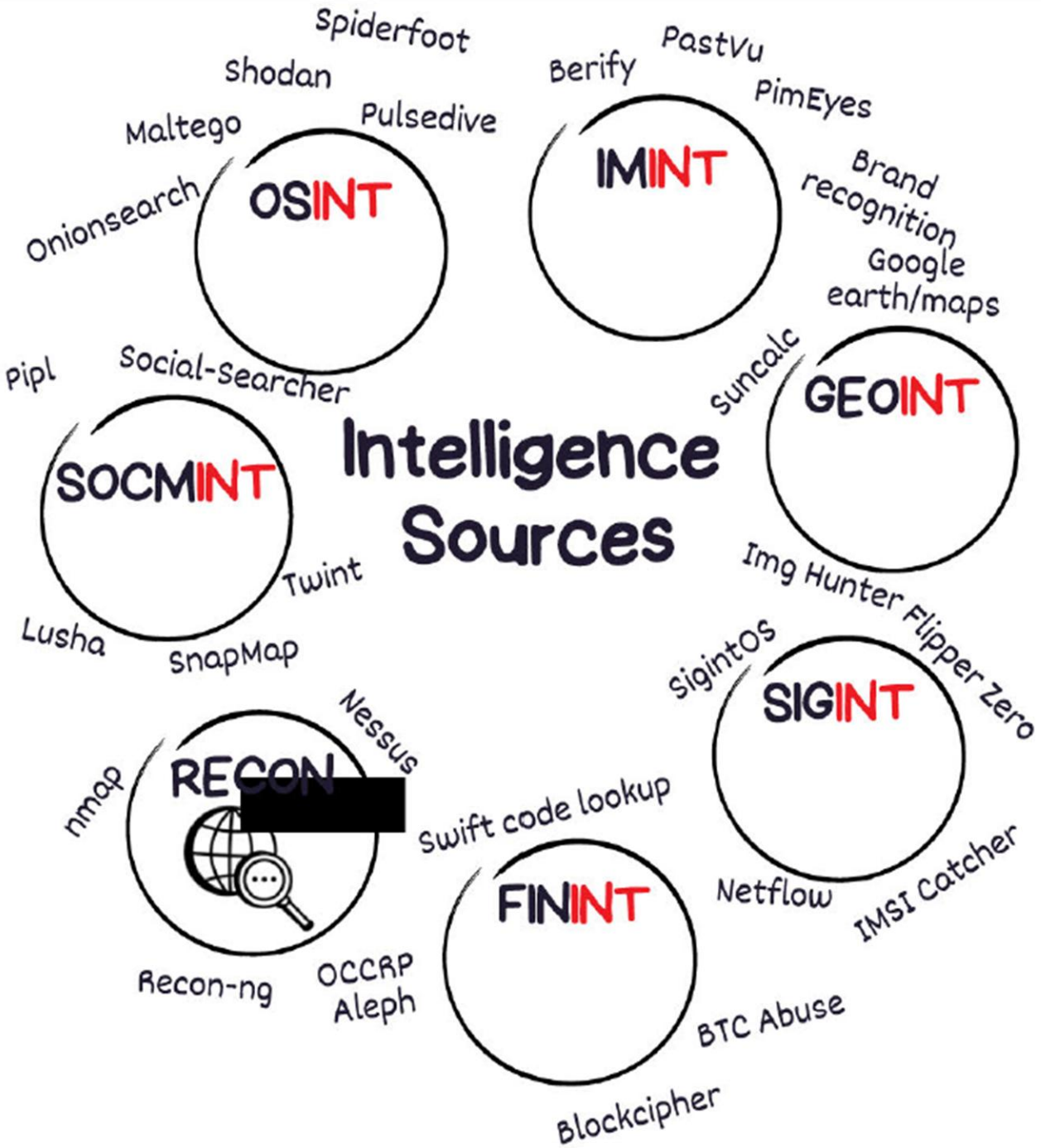
نفوذ را چه چیزی تعریف کنیم؟

- ما دقیقا دنبال کشف چه چیزی هستیم ؟
- این که نفوذ را چه چیزی تعریف کنیم در عکس العمل ما موثر است ؟
  - اگر نفوذ را صرفا یک فایل مشابه فایل آلوده تعریف کنیم؟
  - اگر نفوذ را یک آنومالی در یک مقدار آستانه آماری تعریف کنیم؟

- در این صورت درباب موارد زیر چه می شود کرد؟
  - آیا هر وقوع از مواردی که نفوذ در نظر گرفتیم یک نفوذ مجزاست ؟ ارتباط این موارد به هم چیست؟
  - روایت از حمله هیچ زمینه(context) خاصی را برای ارتباط دادن وقایع دارد؟
  - روندی واکنشی صرف که دربرابر رفتارهای مشخصی عکس العمل دارد. آیابا این تعریف قبل از وقوع رخداد امنیتی میتوان عکس العملی داشت.
  - آیا انتساب حمله ساده است ؟

زنجیره کشتار





انواع جمع آوری اطلاعات

- آیا برای جمع آوری اطلاعات ابزار وجود دارد؟

نگاشت به زنجیره کشتار

۱. شناسایی : مهاجمان متوجه شدند که شرکت‌های فعال در اوکراین برای ثبت اظهارنامه و تبادلات مالیاتی ملزم به استفاده از نرم‌افزار M.E.Doc هستند. به جای نفوذ مجزا به هزاران شرکت و نهاد مختلف، شناسایی روی سرورهای به‌روزرسانی و توسعه شرکت Intellect Serviceتوسعه‌دهنده M.E.Docمتمرکز شد تا یک نقطه نفوذ همگانی پیدا شود.

۲. سلاح‌سازی: مهاجمان کدی به نام بک‌دور را درون یکی از فایل‌های اصلی آپدیت (ماژول جاسازی کردند. این (ZvitPublished.dll کد درون بسته رسمی آپدیت ادغام شد، به طوری که ظاهر یک نسخه معتبر به‌روزرسانی نرم‌افزاری را داشت و امضاهای متعارف بسته را دارا بود.

۳. تحویل: انتقال به قربانی اول: سرورهای رسمی که از پیش تسخیر شده بودند، M.E.Docآپدیت بسته به‌روزرسانی آلوده را از بستر اینترنت به سمت کلاینت‌های مستقر در شبکه‌های سازمانی هدف دند. فایروال‌ها و ابزارهای امنیتی به این ترافیک مشکوک نشدند، چرا که دانلود از دامنه رسمی به‌روزرسانی نرم‌افزار انجام می‌گرفت.

۴. اکسپلویت / بهره‌کشی: در حملات متعارف، این مرحله معمولاً بهره‌برداری از یک حفره حافظه یا آسیب‌پذیری سیستمی است؛ اما در حملات زنجیره تأمین، سوءاستفاده از مکانیسم اعتماد و اجرای خودکار ((Trust Exploitationروی می‌دهد. پروسه قانونی برنامه M.E.Doc بسته را باز کرده و توابع ماژول آلوده را در چارچوب اختیارات خود اجرا کرد (بدون اینکه نیازی به اکسپلویت کرنل یا باگ ویندوزی در این گام باشد).

۵. نصب و استقرار: کد دستکاری‌شده در پوشه برنامه‌ها یا متغیرهای مربوط به نرم‌افزار قرار گرفت و به‌صورت یک مؤلفه فعال ثبت شد. سیستم به‌گونه‌ای پیکربندی شد که در پس‌زمینه سیستم اجرا شود و بتواند پردازه‌های دیگر یا فایل‌های پی‌لود بعدی را راه‌اندازی کند.

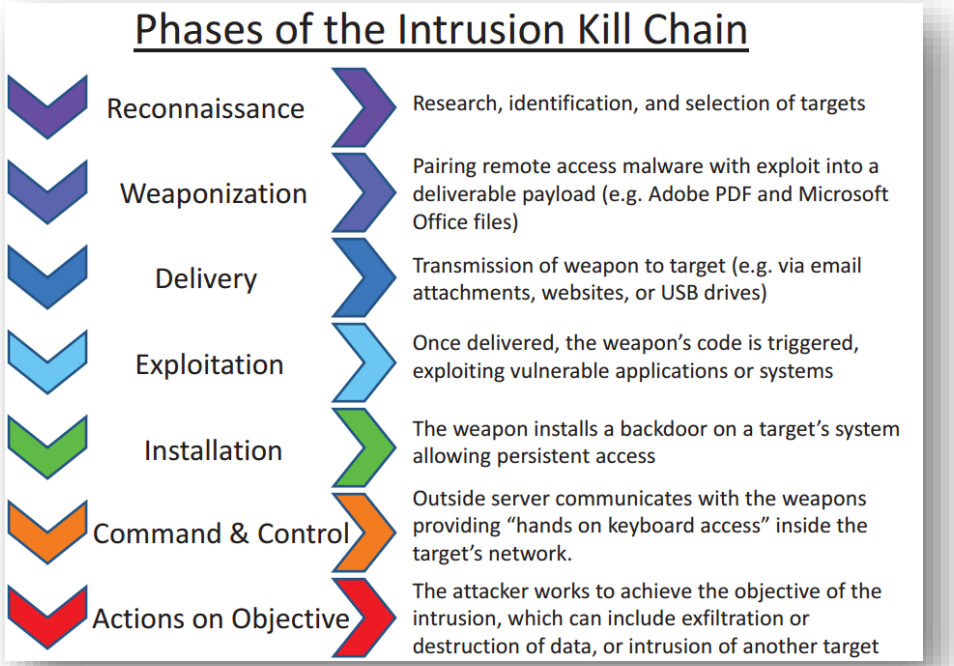
۶. فرماندهی و کنترل : خود بدافزار اصلی نات‌پتیا در فاز نهایی طوری طراحی شده بود که برای تخریب وابسته به یک کانال زنده و طولانی C2بیرونی نباشد (تا بدون وابستگی به اینترنت حتی در شبکه‌های ایزوله بتواند تخریب کند). بنابراین، در چرخه اول، مکانیزم فراخوانی و بارگذاری نهایی از بستر به‌روزرسانی سرور، نقش تحویل دستور نهایی را ایفا کرد.

۷. اقدامات برای دستیابی به اهداف : با اجرای بار نهایی نات‌پتیا روی سیستم اول، اهداف عملیاتی مهاجمان به دو بخش تقسیم شد:

- استخراج اعتبارنامه‌ها و شناسایی داخلی: دامپ کردن هش‌ها و پسوردهای موجود در حافظه رم جهت آماده‌سازی حرکت بیشتر رویشبکه. و یا استفاده از EternalBlueو دیگر پروتکل‌ها برای یافتن میزبان‌های دارای پورت ۴۴۵ در شبکه محلی و پخش سراسری بدافزار.
- تخریب داده‌ها : رمزگذاری فایل‌ها و دستکاری MBR/MFTدیسک به نیت از کار انداختن همیشگی سیستم و بالا نیامدن ویندوز.

نگاشت به زنجیره کشتار

- دور دوم چرخه را می توانید بنویسید؟
- اولین تکلیف



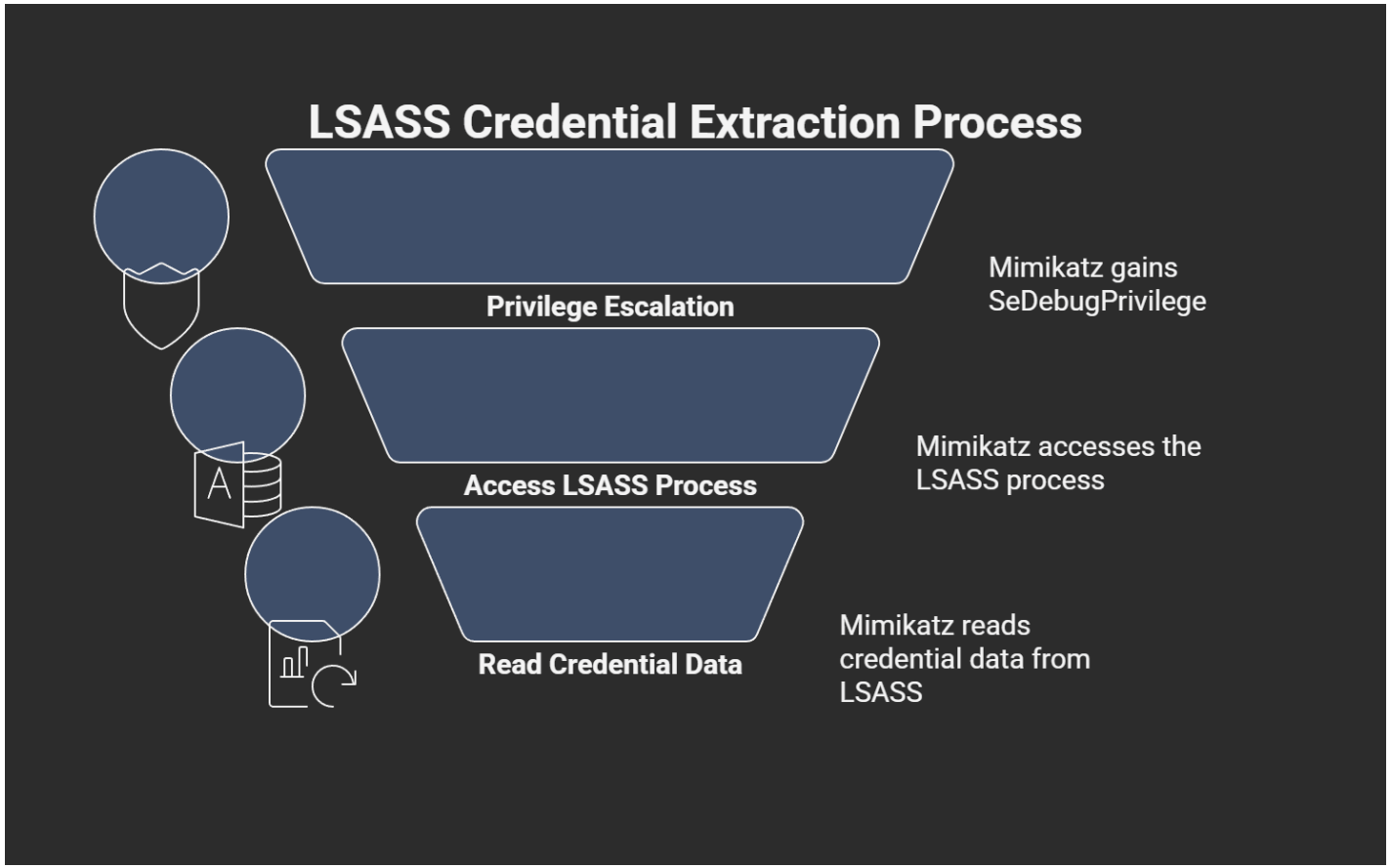
مسیر حمله

- مگر هر سیستم پسورد متفاوتی ندارد؟
- Mimikatzچطور پسوردِ سیستمی را که هنوز واردش نشده از روی یک سیستم دیگر می‌خواند؟
- علت آن ساختار اکتیو دایرکتوری و دامنه‌های ویندوز است:
- در محیط‌های شرکتی، هزاران کامپیوتر عضو یک شبکه واحد به نام Domain هستند.
- مدیران شبکه دائماً برای پشتیبانی، آپدیت یا نصب نرم‌افزار، به سیستم‌های کاربران عادی لاگین می‌کنند (از طریق Remote Desktopیا دستورات ریموت).
- وقتی یک ادمین شبکه روی سیستم کارمند الف لاگین می‌کرد، اعتبارنامه و هش پسورد ادمین در حافظه پروسه lsass.exeآن سیستم باقی می‌ماند.
- حالا سیستم اول که آلوده می‌شد، بدافزار با Mimikatzسراغ حافظه می‌رفت و اکانت ادمین دامنه (Domain Adminیا اکانت محلی یکسانی را که روی تمام سیستم‌های شرکت ادمین بود، استخراج می‌کرد!
- با داشتن پسورد/هش ادمین، بدافزار دیگر نیازی به اکسپلویت کردن باگ نداشت؛ با همان دسترسی قانونی و از طریق ابزارهایی مثل WMIیا PsExecمی‌گفت: سلام سیستم ‘ب’، من مدیر سیستم هستم، این فایل را اجرا کن! و سیستم دوم بی‌چون‌وچرا اجرا می‌کرد.تنها اگر به هر دلیلی اعتبارنامه مناسبی وجود نداشت یا حساب ادمین کار نمی‌کرد، آن وقت سراغ شلیک باگ‌های EternalBlueیا EternalRomanceمی‌رفت.





گربه ملوس MIMIKATZ

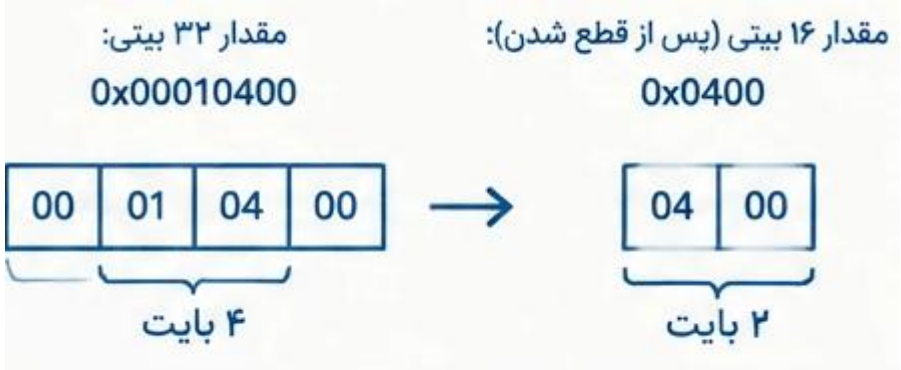


- Mimikatzابزاری امنیتی است که در صورت داشتن دسترسی کافی، می‌تواند برخی داده‌های احراز هویت موجود در حافظه ویندوز—از جمله داده‌های مرتبط با فرایند LSASS، مثل هش‌های NTLMیا بلیت‌های Kerberos—را استخراج کند.
- اینکه رمز عبورِ قابل‌خواندن در دسترس باشد، به نسخه و تنظیمات ویندوز بستگی دارد و تضمینی نیست؛
- قابلیت‌هایی مانند LSA Protectionو Credential Guardهم می‌توانند دسترسی به این داده‌ها را محدود کنند.

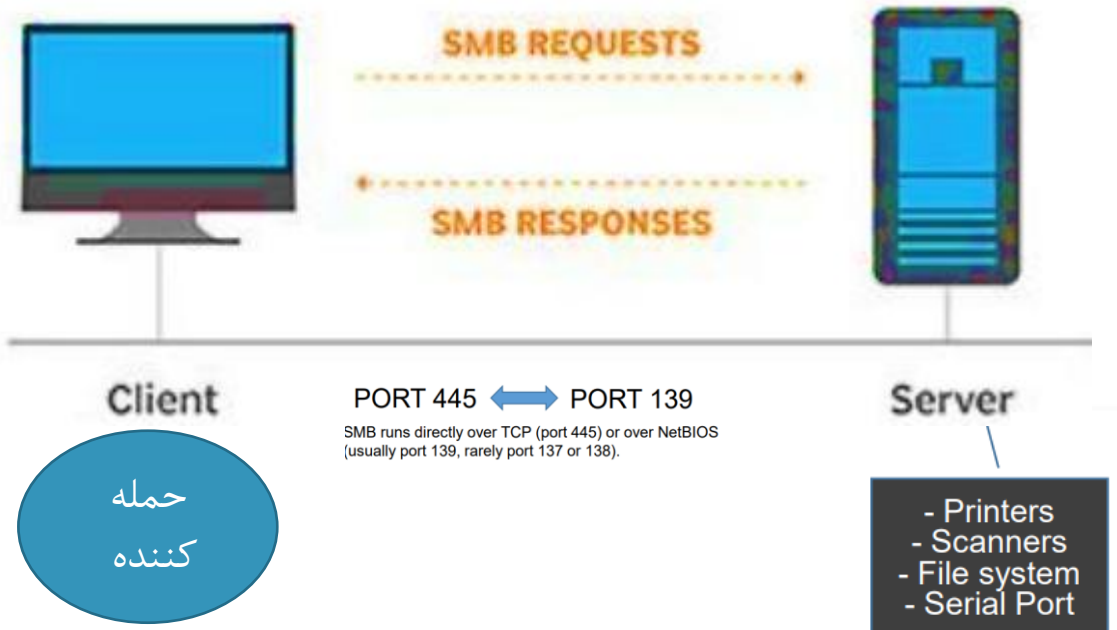
این گربه چگونه پیدا شد؟

- خالق میمیکتز بعدها در کنفرانس‌ها اعلام کرد که انگیزه، اولیه او کشف یک ابزار مخرب نبود، بلکه صرفاً می‌خواست زبان برنامه‌نویسی Cرا عمیق‌تر یاد بگیرد و با نحوه، تعامل ویندوز با حافظه (RAM)تمرین کند.
- در جریان این یادگیری، او به بررسی پروسه، احراز هویت ویندوز (lsaas.exe)و به‌ویژه ماژول قدیمی WDigestپرداخت. دلفی متوجه شد که ویندوز برای انجام احراز هویت سریع ((SSO، گذرواژه، رمزگذاری‌شده را در حافظه نگه می‌دارد. اما نکته، شوکه‌کننده این بود که کلید رمزگشایی آن گذرواژه نیز درست در همان حافظه و در همان پروسه کنار آن نگه‌داری می‌شد!
- به تعبیر خود دلفی: این کار شبیه این بود که یک فایل رمزگذاری‌شده محرمانه را داخل ایمیلی بگذارید و رمز فایل را هم در متن همان ایمیل بنویسید!
- دلفی این یافته را در سال ۲۰۱۱ به مایکروسافت گزارش کرد؛ اما پاسخ مایکروسافت در آن زمان این بود که برای خواندن رم یک سیستم، فرد باید از قبل دسترسی ادمین روی آن سیستم داشته باشد، پس این یک آسیب‌پذیری بحرانی تلقی نمی‌شود.
- دلفی استدلال کرد که این دیدگاه، خطر حرکت جانبی را نادیده می‌گیرد؛ یعنی مهاجم با گرفتن یک ماشین معمولی در شبکه، می‌تواند رمز مدیر کل شبکه را از حافظه بیرون بکشد و بقیه شبکه را تصاحب کند.
- عرضه و متن‌باز شدن: دلفی برای اثبات ادعایش، کد ابزار را نوشت و آن را ابتدا به شکل فایل باینری منتشر کرد. سپس در سال ۲۰۱۲، پس از یک کنفرانس امنیتی در مسکو و احساس ناامنی نسبت به سرقت کدش، سورس‌کد کامل میمیکتز را روی گیت‌هاب متن‌باز کرد تا همگان و مدافعان امنیت بتوانند نحوه عملکرد و راه‌های دفاع در برابر آن را ببینند.

سرریز به روش بلوایترنال



- وقتی شما به پورت ۴۴۵ وصل می‌شوید، قبل از این‌که ویندوز به شما بگوید: *یوزرنیم و پسوردت را بگو تا احراز هویتت کنم*، ابتدا باید یک مرحله توافق اولیه پروتکل انجام شود (مثل پروتکل دست دهی).
- آسیب‌پذیری **EternalBlue** دقیقاً در همین مرحله اولیه قبل از لاگین یا **Pre-Authentication** و مستقیماً داخل درایور کرنل srv.sys رخ می‌داد.
- بنابراین، پورت ۴۴۵ باز بود چون جزو ذات ویندوز شبکه‌ای است، و سیستم هدف هم جواب می‌داد چون فکر می‌کرد دارد یک بسته معمولی شبکه دریافت می‌کند؛ غافل از اینکه پکت ارسالی عمداً به گونه‌ای دستکاری شده بود که کرنل ویندوز را سرریز کند و بدون هیچ لاگینی، کد را مستقیماً در مغز سیستم‌عامل اجرا نماید





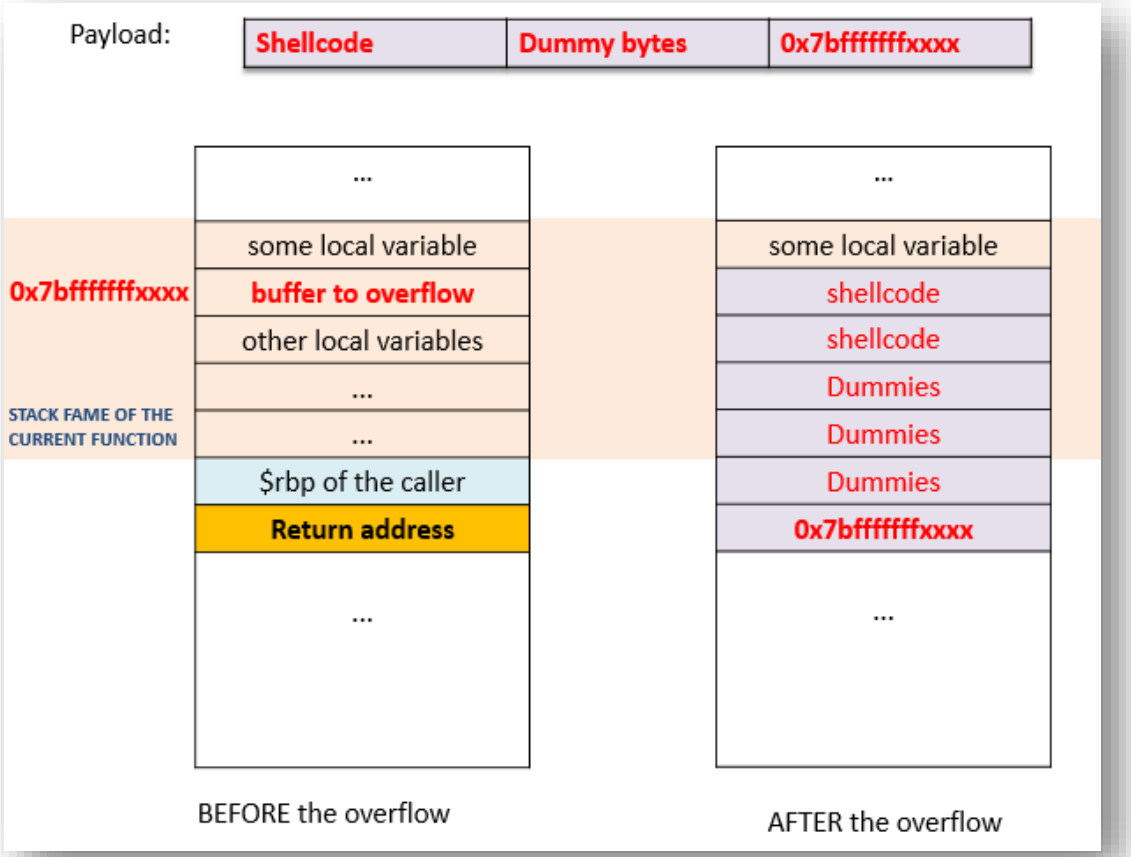
سرریز بافر کرنل در اینترنال بلو

- صحنه: شبکه محلی سازمان، پورت ۴۴۵ باز است. ویندوز قربانی پشت میزش نشسته و مهاجم از طریق یک آپدیت آلوده دارد روی همین سیستم اجرا می شود.
- مهاجم: سلام ویندوز! من یک دستور/سند بزرگ برای سیستم اشتراک فایل و پرینترت دارم، چون بزرگه می‌خوام تو چند تکه بفرستمش.
- ویندوز (پروتکل SMB) سلام! اوکی، بفرست. فقط بگو در مجموع کلاً چقدر حجم داره تا برم داخل رم (RAM) برات جا رزرو کنم؟
- مهاجم: ارسال ترفند ۳۲ بیتی NT_TRANSACT

  اندازه کلش دقیقاً ۶۶،۵۶۰ بایت است (به زبان کامپیوتر: x00010400
- ویندوز: (دچار باگ ریاضی می‌شود؛ چشمانش ۱۶ بیتی است و فقط نیمه دوم عدد را می‌خواند!) آها دیدم! گفتی اندازه کلش ۱۰۲۴ بایت (۱ کیلوبایت) است x0400 .من رفتم یه ظرف کوچیک ۱ کیلوبایتی تو امن‌ترین جای حافظه هسته برات رزرو کردم. حالا بقیه‌ش رو بفرست!
- مهاجم: بفرما! این هم کل محموله! (یکجا ۶۶ کیلوبایت داده مخرب شلیک می‌کند)
- ویندوز: (درحال ریختن داده‌ها داخل ظرف) ای وای... یک کیلو ریختم... چرا تموم نمیشه؟! داره سرریز می‌کنه! ریخت روی میز! ریخت روی مغزم! حافظه هسته قاطی کرد... دستورات خودم پاک شد...
- مهاجم: دیر شد! اون ۶۵ کیلوبایت اضافه‌ای که سرریز کرد، کدهای من بود. حالا برو کنار، من از این لحظه ادمین ارشد کل این سیستم هستم!

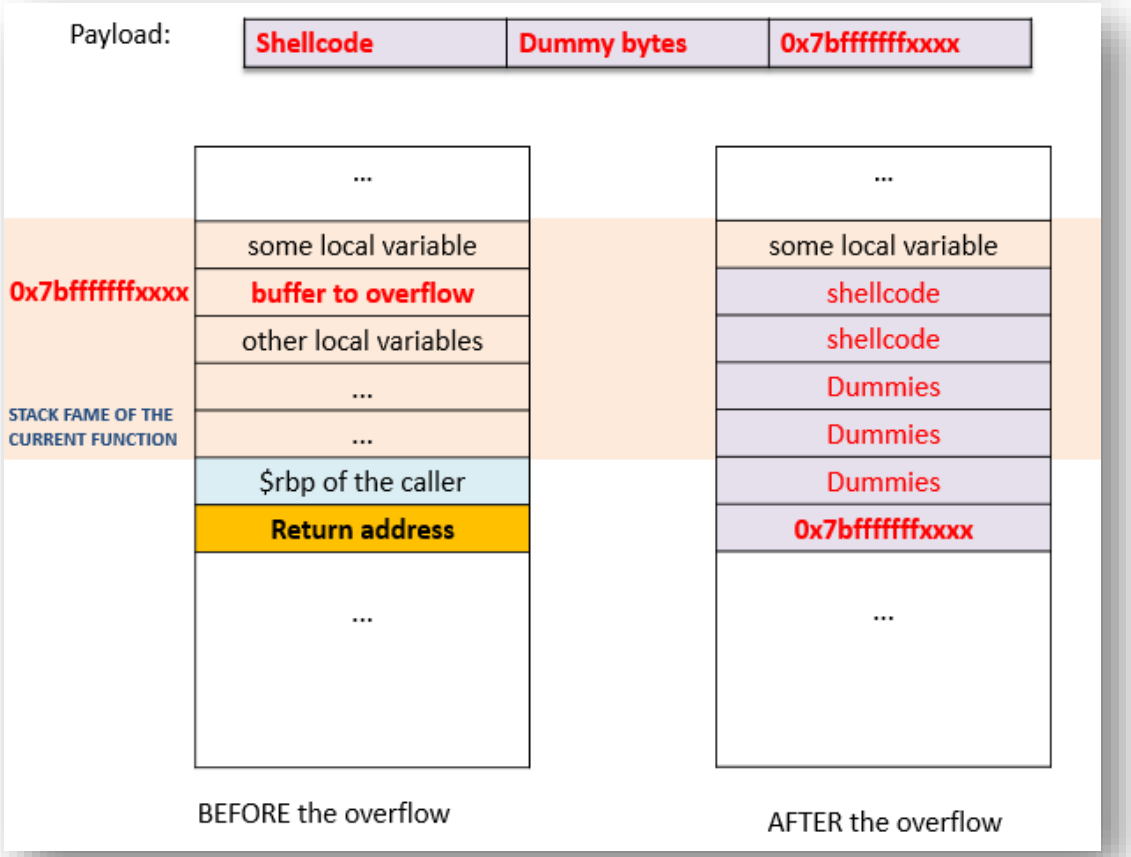
- دنبال یک آسیب پذیری هستیم با دسترسی خواندن به حافظه
- پروتکل SMB درنسخه های اولیه می خواست با سیستم عامل های قبل از ویندوز همخوانی داشته باشد پس یک قابلیت فایل که در آن سیستم عامل کاربرد داشت را دردرایور srv.sys هم تعریف کرد که عبارت بود از ویژگی های یک فایل.
- تبدیل FEA (file’s extended attributes) غیر ویندوز به ویندوز ویژگی کم کاربرد وکمترتست شده ای بود که در صورت اعلام سایز بزرگ فایل سبب سرریز عدد صحیح (integer overflow) می شد.
- این سرریزسبب تخصیص یک بافر خیلی کوچک می شد.
- یک مقدار بزرگ را در یک مقدار کوچک تخصیص یافته کپی می کند.
- بقیه حجم خواسته شده از کرنل کپی می شوند و در پاسخ قرار می گیرند و سبب افشای حافظه می شوند .

مسیر تکاملی نفوذهای نرم افزاری–گام اول





مسیر تکاملی نفوذهای نرم افزاری



سرریز بافر

Data Execution Prevention (DEP/NX):
از اجرای کد در صفحات داده (مانند پشته یا هیپ) جلوگیری می‌کند.

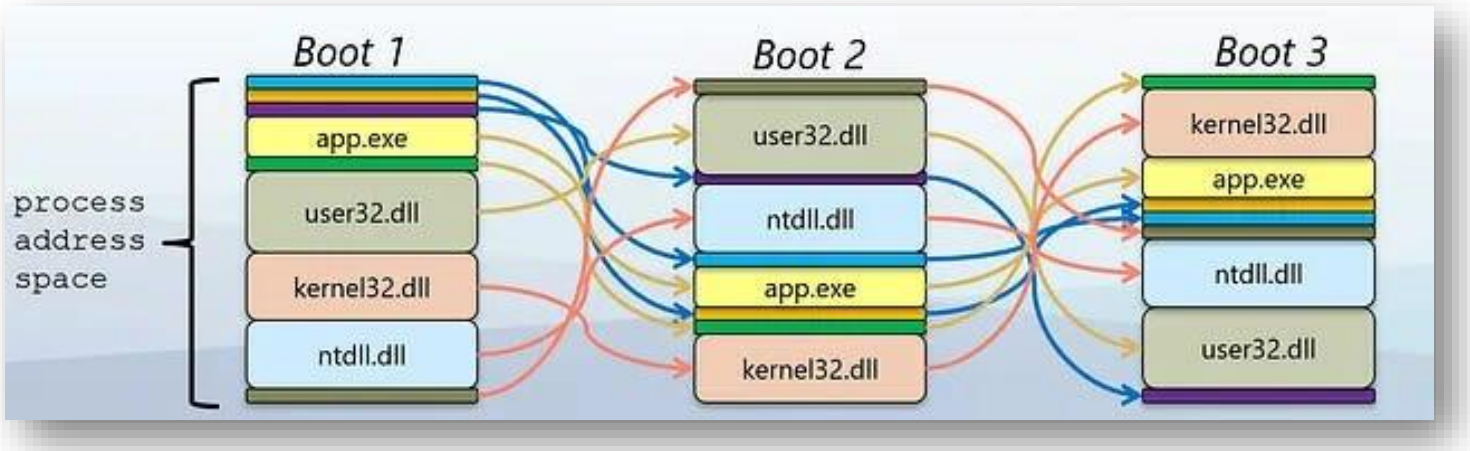
Address Space Layout Randomization (ASLR):
آدرس‌های پایه فایل‌های اجرایی و کتابخانه‌ها را تصادفی می‌کند، بنابراین سرریز شما نمی‌تواند پیش‌بینی کند که به کجا باید پرش کند.



آیا حمله کننده متوقف شد؟

فکر می کنید چطور می شود همچنان با نفوذ در یک برنامه یک کد دلخواه را اضافه کرد ؟

مسیر تکاملی نفوذهای نرم افزاری–گام سوم



- **برای مقابله با ASLR**
- از بخش های ثابت و غیر تصادفی شده کمک بگیریم!
  - برخی بخش های سیستم عامل تصادفی نمی شوند.
  - اگر فایل باینری به عنوان سازگار با ASLRعلامت‌گذاری نشده باشد، سیستم عامل آن را در آدرس پایه ترجیحی خود بارگذاری می‌کند (اغلب x00400000 ۰در ویندوز ۳۲ بیتی).
  - فایل‌های اجرایی exe یا DLLهایی که بدون پرچم پیونددهنده DYNAMICBASE در ویندوز یا بدون fPIE/-pieدر لینوکس کامپایل می‌شوند.
  - در ویندوز: اکثر درایورها به طور پیش‌فرض ASLRفعال نیستند، مگر اینکه با / DYNAMICBASEکامپایل شوند.
  - برخی از مناطق ورودی/خروجی نگاشت شده با حافظه (memory-mapped I/O) یا مناطق سازگار با DOSقدیمی (به عنوان مثال، ۶۴ KB پایین) کلا فیکس طراحی شده اند. .

مسیر تکاملی نفوذهای نرم افزاری–گام دوم

- برای مقابله با ASLR
- از بخش های ثابت و غیر تصادفی شده کمک بگیریم!
  - برخی بخش های سیستم عامل تصادفی نمی شوند
- آدرس پایه را در هنگام حمله بخوانیم!
  - از یک آسیب پذیری افشای حافظه (memory disclosure) استفاده می شود تا آدرس ابتدای یک برنامه پیدا شود.
  - چطورمی توان سبب افشای حافظه دلخواه شد؟
  - در این صورت تا چه زمانی این آدرس های خوانده شده تغییر نمی کنند؟

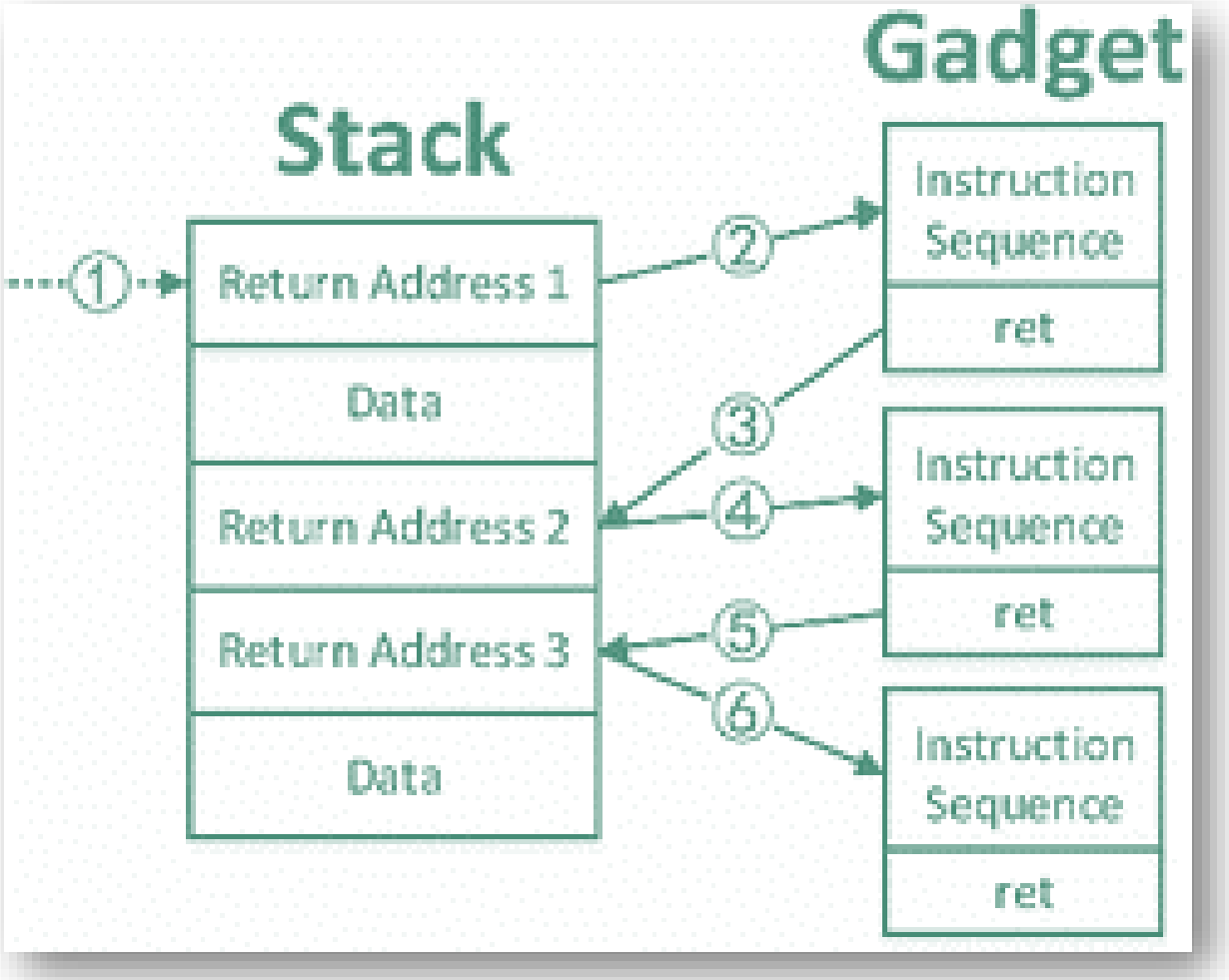
مسیر تکاملی نفوذهای نرم افزاری–گام سوم

- **برای مقابله با DEP**
- بعد از سرریز اولیه و گرفتن اختیار اجرا به دست با بازنویسی آدرس بازگشت، بجای اینکه به نقطه ای که خودمان کد نوشتیم در بافر(یا هیپ) بازگردیم، به یک نقطه از کد اجرایی موجود در خود برنامه بازگشت کنیم که کد مدنظر ما در آن است!!
  - مگر کد مدنظر ما جایی هست ؟
  - اگر تکه تکه و دستور دستور بنویسیم، بله☺
  - پس کافی است کد مدنظر را به دستوراتی تجزیه کنیم که در کد برنامه وجود دارند و بعد از هریک از آنها یک دستور return نیز هست.
  - آدرس همه این دستورات را در پشته قرار می دهیم
  - با بازگشت به آدرس اولین دستور، زنجیره ای از دستوراتی که انتخاب کردیم شروع به اجرا می کند، چون با هر بازگشت یک پاپ از سر پشته انجام می شود و عملا به همان نقطه پرش می شود.
  - به این تاکتیک برنامه نویسی بازگشت–مبنا (return-oriented programming) می گویند.

RETRUN =

1.Read the 8-byte (x64) value at the top of the stack ([RSP]).
2.Set RIP (instruction pointer) to that value → jump there.
3.Increment RSP by 8 (x64) — effectively "popping" the address.





اگر EternalBlue سرریز است، پس ASLR و DEP چطور دور زده شدند؟

چرا EternalBlue به افشای حافظه برای ASLR نیاز نداشت؟

- در بهره‌کشی برنامه های سطح کاربری، برنامه جای تصادفی دارد و شما تا حافظه را نخوانید (Leakنکنید) نمی‌دانید شل‌کد کجاست.
- اما طراحان NSAدر EternalBlueیک کشف نبوغ‌آمیز در کرنل ویندوز داشتند:
- نقطه ضعف سخت‌افزاری/کرنل ویندوزهای هدف یک ناحیه در کرنل دارند به نام HAL (Hardware Abstraction Layer).معلوم شد که آدرس این استخر در کرنل تا قبل از آپدیت‌های ساختاری ویندوز ۱۰، کاملاً ثابت و غیرتصادفی بود:
- اکسپلویت‌نویس اصلاً نیازی نداشت آدرس‌ها را با افشایحافظه کشف کند!
- او دقیقاً می‌دانست که این آدرس همیشه ثابت است. به همین خاطر EternalBlueتنها اکسپلویت خانواده MS17-010بود که به مرحله‌ی افشای حافظه نیازی نداشت.

مسئله DEP

- در نسخه‌های آن زمانِ ویندوز، ناحیه صفحه HAL در کرنل دارای دسترسیِ **اجرایی (Executable)** بود! بنابراین وقتی ارتباط بسته می‌شد، پردازنده مستقیماً به آدرس شل‌کد پرش می‌کرد و کد را اجرا می‌کرد.**نیازی به ساخت زنجیره ROP نبود، چون مستقیماً اشاره‌گر تابعی در ناحیه‌ای اجرا شد که DEP مانع آن نمی‌شد.**

نشانه های نفوذ (Indicators of Compromise)

- نشانه‌های نفوذ، مصنوعات قابل مشاهده‌ای هستند که مربوط به یک مهاجم یا فعالیت‌های او، مانند تاکتیک‌ها، تکنیک‌ها، رویه‌ها و ابزارها و زیرساخت‌های مرتبط با آن، می‌باشند.
- این شاخص‌ها را می‌توان در سطح شبکه یا نقطه پایانی (میزبان) مشاهده کرد و می‌توانند با درجات مختلفی از اطمینان، به مدافعان شبکه کمک کنند تا :
  - به طور پیشگیرانه ترافیک مخرب یا اجرای کد را مسدود کنند،
  - یک نفوذ سایبری رخ داده را تعیین کنند
  - یا فعالیت کشف شده را به یک مجموعه نفوذ شناخته شده مرتبط سازند.

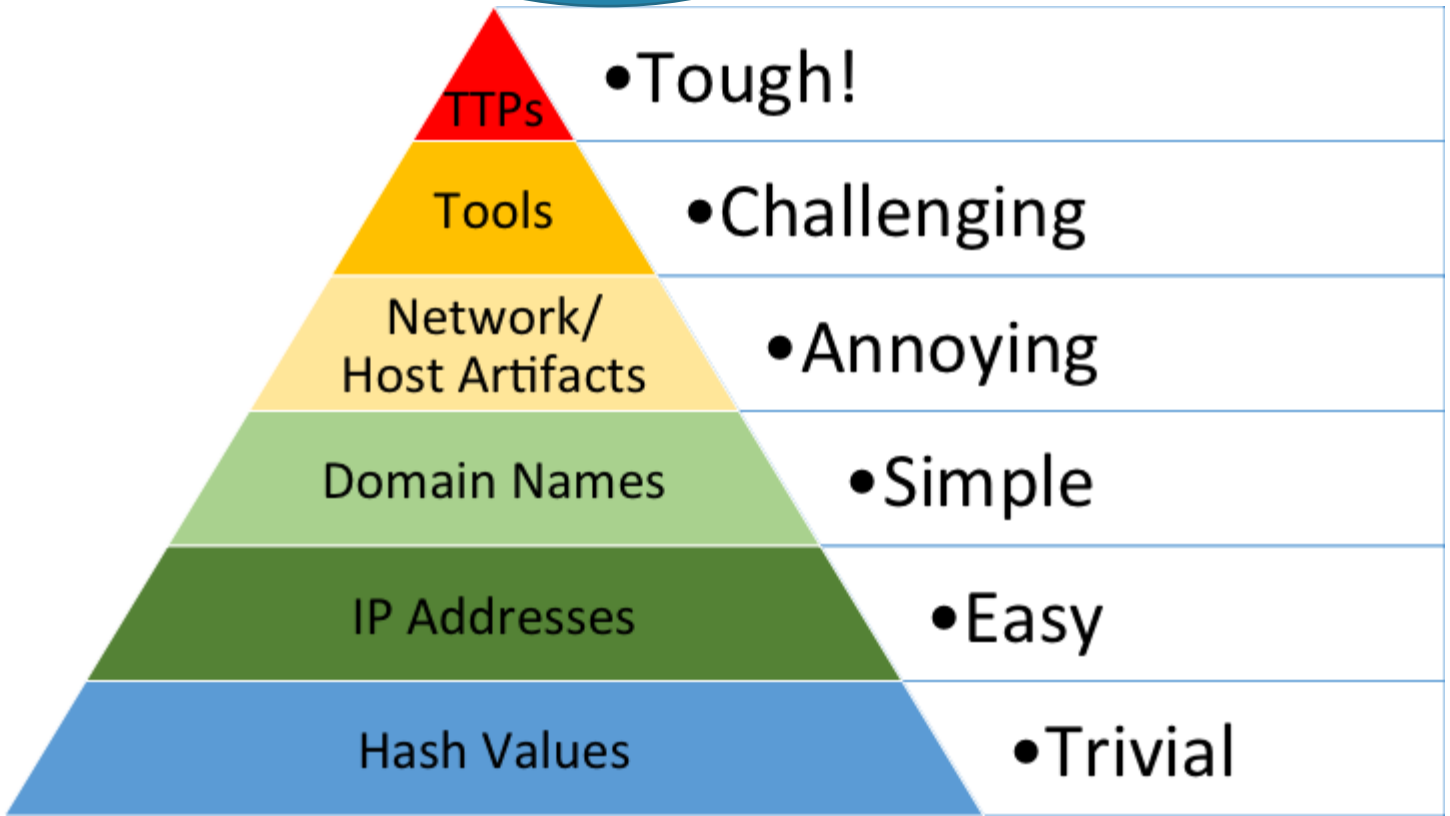


فواید IOC

- IoC ها لایه های چندگانه استراتژی دفاع در عمق مدرن را پشتیبانی و فعال می کنند.
- IoCها می توانند حتی با منابع محدود استفاده شوند.
- IoCها تاثیر مضاعفی بر تلاش های دفاعی حمله در یک سازمان دارند.
- IoCها به راحتی بین سازمان ها به اشتراک گذاشته می شوند.
- IoCها می توانند صرفه جویی قابل توجهی در زمان داشته باشند.
- IoCها امکان کشف حملات تاریخی را فراهم می کنند.
- IoCها را می توان به تهدیدات خاص نسبت داد.

هرم درد (Pyramid of pain)

چرا هرم درد؟
مهاجم برای تغییر این مصنوعات
چه میزان درد را تجربه می کند؟



•در رأس هرم، **TTPها و متدولوژی مهاجم** قرار دارد؛ یعنی اگر جلوی رفتاری مثل دسترسی به *LSASS* یا حرکت عرضی با *PsExec* را بگیرید، مهاجم مجبور است کل استراتژی و دانش خود را از اول طراحی کند (بیشترین درد و هزینه برای مهاجم).

Mitre TTP

تاکتیک‌ها چرایی یک تکنیک یا زیرتکنیک ATT&CK را نشان می‌دهند.
آن‌ها اهداف فنی دشمن، دلیل انجام یک عمل و آنچه را که سعی در دستیابی به آن دارند، بیان می‌کنند. به عنوان مثال، یک دشمن ممکن است بخواهد برای دسترسی به یک شبکه هدف، به دسترسی به اعتبارنامه دست یابد. هر تاکتیک شامل مجموعه‌ای از تکنیک‌هایی است که مدافعان شبکه مشاهده کرده‌اند که توسط بازیگران تهدید به صورت گسترده استفاده می‌شوند.

تکنیک‌ها نشان دهنده چگونگی دستیابی یک دشمن به یک هدف تاکتیکی با انجام یک عمل هستند. به عنوان مثال، یک دشمن ممکن است اعتبارنامه‌ها را برای دستیابی به دسترسی به اعتبارنامه‌ها حذف کند. تکنیک‌ها همچنین ممکن است نشان دهنده آنچه یک دشمن با انجام یک عمل به دست می‌آورد، باشند. یک تکنیک یک رفتار خاص برای دستیابی به یک هدف است و اغلب یک گام واحد در یک رشته فعالیت‌ها است که برای تکمیل ماموریت کلی دشمن در نظر گرفته شده است.

رویه‌ها نشان دهنده چیزی که یک دشمن انجام داده است و نمونه‌هایی از نحوه استفاده یک دشمن از یک تکنیک یا زیرتکنیک هستند. برای مثال، رویه‌های مختلفی برای حذف اعتبارنامه‌های سیستم عامل وجود دارد: حافظه LSASS [T1003.001] که مبتنی بر استفاده از ابزارها، امکانات و دستورات مختلف است. دانستن رویه‌ها می‌تواند برای تکرار یک حادثه با شبیه‌سازی دشمن و برای تشخیص فعالیت‌های مخرب مفید باشد.

Tactic

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Exfiltration	Command and Control
Drive-by Compromise	AppleScript	.bash_profile and .bashrc	Access Token Manipulation	Access Token Manipulation	Account Manipulation	Account Discovery	AppleScript	Audio Capture	Automated Exfiltration	Commonly Used Port
Exploit Public-Facing Application	CMSTP	Accessibility Features	Accessibility Features	BITS Jobs	Bash History	Application Window Discovery	Application Deployment Software	Automated Collection	Data Compressed	Communication Through Removable Media
Hardware Additions	Command-Line Interface	AppCert DLLs	AppCert DLLs	Binary Padding	Brute Force	Browser Bookmark	Distributed Component	Clipboard Data	Data Encrypted	Connection Proxy
Replication Through Removable Media	Control Panel Items	AppInit DLLs	AppInit DLLs	Byp... Acc...					Data Transfer Size Limits	Custom Comm... Co...
									Exfil...	

Technique

Drive-by Compromise

A drive-by compromise is when an adversary gains access to a system through a user visiting a website over the normal course of browsing. With this technique, the user's web browser is targeted for exploitation. This can happen in several ways, but there are a few main components:

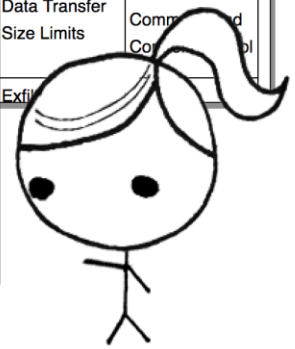
Multiple ways of delivering exploit code to a browser exist, including:

- A legitimate website is compromised where adversaries have injected some form of malicious code such as JavaScript, iFrames, cross-site scripting.
- Malicious ads are paid for and served through legitimate ad providers.
- Built-in web application interfaces are leveraged for the insertion of any other kind of object that can be used to display web content or contain a script that executes on the visiting client (e.g. forum posts, comments, and other user controllable web content).

Often the website used by an adversary is one visited by a specific community, such as government, a particular industry, or region, where the goal is to compromise a specific user or set of users based on a shared interest. This kind of targeted attack is referred to a strategic web compromise or watering hole attack. There are several known examples of this occurring.[1]

Drive-by Compromise	
Technique	
ID	T1189
Tactic	Initial Access
Platform	Linux, Windows, macOS
Permissions Required	User
Data Sources	Packet capture, Network device logs, Process use of network, Web proxy, Network intrusion detection system, SSL/TLS inspection

Procedure



منطق کشف و تشخیص

- فرض کنید یک بانک مورد حمله قرار گرفته و شما مسئول این هستید که به سایر بانک ها کمک کنید گرفتار حمله مشابه نشوند.
- یک راه انتقال نشانه های نفوذ از طریق فرمت های استاندارد است. اینکه این فرمت چه است خیلی ربطی به بحث فعلی ندارد. این بحث درباره منطق کشف است.
- فرمت اشتراک‌گذاری (مثل STIX) یک جعبه یا پاکت نامه استاندارد است. این ظرف صرفاً اطلاعات متاداده دارد:فرستنده کیست؟ تاریخ کشف چه زمانی بوده؟ سطح خطر چقدر است؟ این تهدید متعلق به کدام گروه هکری است؟
- ما دنبال محتوای داخل جعبه هستیم؛ یعنی آن سرنخ یا منطقی که داخل پاکت می‌گذارید و برای همکار یا سازمان دیگر می‌فرستید.
- فرض کنید مهاجم می‌خواهد با ابزاری حافظه پروسه lsass.exe را بخواند تا هش پسوردهای شبکه را به دست بیاورد.
-

سطح ۱: رویکرد ساده‌لوحانه — هش فایل

•**داده انتقال‌یافته:** در یک فید STIX، هش ۲۵۶SHA ابزارِ رسمی Mimikatz به دست شما می‌رسد:۹۲e۸...۵f۳a

•**مکانیزم دفاع:** فایروال یا آنتی‌ویروس این هش را در بلاک‌لیست خود می‌گذارد.

•**حرکت مهاجم:** مهاجم به سورس‌کد پروژه یک کامنت خالی یا چند بایت تصادفی اضافه کرده و دوباره کامپایل می‌کند.

•**نتیجه:** هش کاملاً عوض می‌شود؛ تمام سامانه‌های مبتنی بر انتقالِ صرف هش دور زده می‌شوند. **(درد مهاجم: صفر ثانیه!)**

- **سطح ۲: ارتقا به YARA تشخیص محتوایی ابزار (Tool / Memory Inspection)**
- تعریف لاجیک کشف: ما نمی‌دانیم مهاجم چه هشی می‌سازد، اما می‌دانیم داخل باینری ابزار Mimikatzتوابع و ساختارهای خاصی برای خواندن رم وجود دارد (مثل فراخوانی توابع lsasrv.dllیا آرگومان‌های مشهورش.
- موتور YARAروی هارد دیسک فایل را اسکن می‌کند یا به رم سیستم متصل شده و بایت‌به‌بایت حافظه پروسه‌ها را می‌گردد. فرقی نمی‌کند نام فایل چیست یا هشش چیست؛ اگر آن رشته‌ها و پترن‌های بایت در رم باشد، کشف می‌شود.
- حرکت مهاجم: مهاجم با روش‌های پیشرفته (Obfuscation / Memory Encryption)، تمام رشته‌ها را رمزنگاری می‌کند یا اصلاً از Mimikatzاستفاده نمی‌کند؛ خودش یک اسکریپت ساده C++می‌نویسد که از صفر فقط با توابع داخلی ویندوز به LSASSوصل شود. اینجا YARAهم ممکن است دور بخورد.

- **سطح ۳: سقف هرم درد با Sigma تشخیص رفتار و سناریو (TTPs / Event Logs)**
- تعریف لاجیک کشف: مهاجم ابزارش را عوض کرد، رشته‌ها را رمز کرد، هش را عوض کرد.
- اما یک حقیقت قطعی در معماری ویندوز وجود دارد:هر برنامه‌ای بخواهد پسوردها را بخواند، سیستم‌عامل ثبت می‌کند که پروسه Xتلاش کرده با دسترسی خواندن حافظه (PROCESS_VM_READ = 0x10)به پروسه lsass.exeمتصل شود.این رفتار غیرقابل انکار و تغییرناپذیر است.

Yara در مقابل سیگما

```yara
rule Mimikatz_Sekurlsa_Strings {
    meta:
        description = "Detects mimikatz strings in binary or process memory"
    strings:
        $str1 = "sekurlsa::logonpasswords" ascii wide
        $str2 = "lsasrv.dll" ascii wide nocase
        $hex_pattern = { 89 45 FC 8B 45 FC 89 85 }
    condition:
        2 of ($str*) or $hex_pattern
}
```

```yaml
title: LSASS Memory Access by Non-System Process
status: experimental
logsource:
    category: process_access
    product: windows
detection:
    selection:
        TargetImage|endswith: '\lsass.exe'
        GrantedAccess|contains: '0x10' # PROCESS_VM_READ
    filter:
        SourceImage|startswith: 'C:\Windows\System32\' # پروسه های مجاز خود سیستم
    condition: selection and not filter
```

در ۲۰۲۶ هستیم و TTPهم دردناک ترین سطح هرم درد نیست .

- وقتی David Bianco در سال ۲۰۱۳ هرم درد را معرفی کرد، رأس هرم را TTPsقرار داد و گفت کشف و سد کردن این لایه بیشترین درد و هزینه را به مهاجم تحمیل می‌کند .
- اما با گذشت بیش از یک دهه، مشخص شده که خود لایه TTPدچار فرسایش مفهومی شده است.
- به نظر شما چرا؟

بخش اول: نقص‌های ذاتی و ساختاری TTP

- **۱. نگاه اتمیک به جای پیوستار رفتاری**
  - **مشکل:** مدل TTP یک پدیده پیوسته و دینامیک (حمله) را به رویدادهای گسسته و مستقل اتمی تکه‌تکه می‌کند.
  - **پیامد:** در واقعیت، یک حمله حاصل برهم‌کنش این قطعات است، نه وجود تک‌تک آن‌ها. وقتی رفتارها اتمیک دیده می‌شوند، ترکیب دو رفتار کاملاً عادی و قانونی در کنار هم می‌تواند یک فاجعه امنیتی بسازد، اما در هیچ‌یک از خانه‌های TTP به عنوان رفتار مخرب شناسایی نشود (ترکیب‌پذیری نادیده گرفته می‌شود).
- **۲. مسطح بودن و فقدان ساختار مرتبه‌بالا**
  - **مشکل:** در ماتریس‌هایی مثل ATT&CK، رابطه بین تاکتیک‌ها و تکنیک‌ها یک درخت با عمق بسیار کم است
  - **پیامد:** این ساختار فاقد **گرامر رابطه‌ای** است. شما نمی‌توانید گزاره‌های شرطی، احتمالی و بازگشتی بسازید.
  - مدل TTP یک واژه‌نامه است، نه یک زبان. با واژه‌نامه نمی‌توان داستانِ حمله را خواند.
- **۳. تکیه بر نحوه انجام به‌جای ماهیت اثر**
  - **مشکل:** TTPها ذاتاً عمل‌محور هستند (مثلاً: تکنیک فلان چگونه فایل را رمز می‌کند؟).
  - **پیامد:** اما امنیت در سطح نظری یعنی **حفظ نامتغیرها**. اینکه مهاجم با APIهای ویندوز، با کدهای اسمبلی خام، یا از طریق یک آسیب‌پذیری روز-صفر در درایور، دیتای حافظه را تغییر می‌دهد، تفاوت در روال کار است؛ اثر نهایی یکی است: نقض محرمانگی حافظه. مدل TTP چون روی نحوه انجام تمرکز دارد، با هر روش جدید غافلگیر می‌شود.

بخش دوم: نقص‌های پیاده‌سازی و ابزاری TTP

این نقص‌ها ناشی از تنزل دادن این مفهوم توسط صنعت امنیت، ابزارهای EDR/SIEM و مهندسان تشخیص است:

۱ .سقوط سمانتیک به سطح سینتکس

•**مشکل:** قرار بود TTP بالاترین لایه باشد (بررسی الگوهای رفتاری کلان)، اما فروشندگان امنیتی و طراحان رول‌ها برای پیاده‌سازی آن در نرم‌افزار، آن را به **رجکس (Regex)، نام فایل، یا هشدارهای EventID** تبدیل کردند.

•**پیامد:** اجرای T1059.001 (پاورشل) در عمل به معنای مانیتور کردن پروسه‌ای به نام powershell.exe پیاده‌سازی شد! مهاجم کافی است فایل اجرایی را تغییر نام دهد، یا از کدهای Unmanaged دات‌نت استفاده کند تا از فیلتر این رول فرار کند بدون اینکه در ماهیت رفتارش تغییری ایجاد شده باشد.

۲ .توهم پوشش امنیتی

•**مشکل:** ماتریس TTP به شکل یک جدول اکسل طراحی شده و مدیران امنیت تصور می‌کنند اگر خانه‌ها را در داشبورد خود «سبز» کنند، امنیت برقرار است.

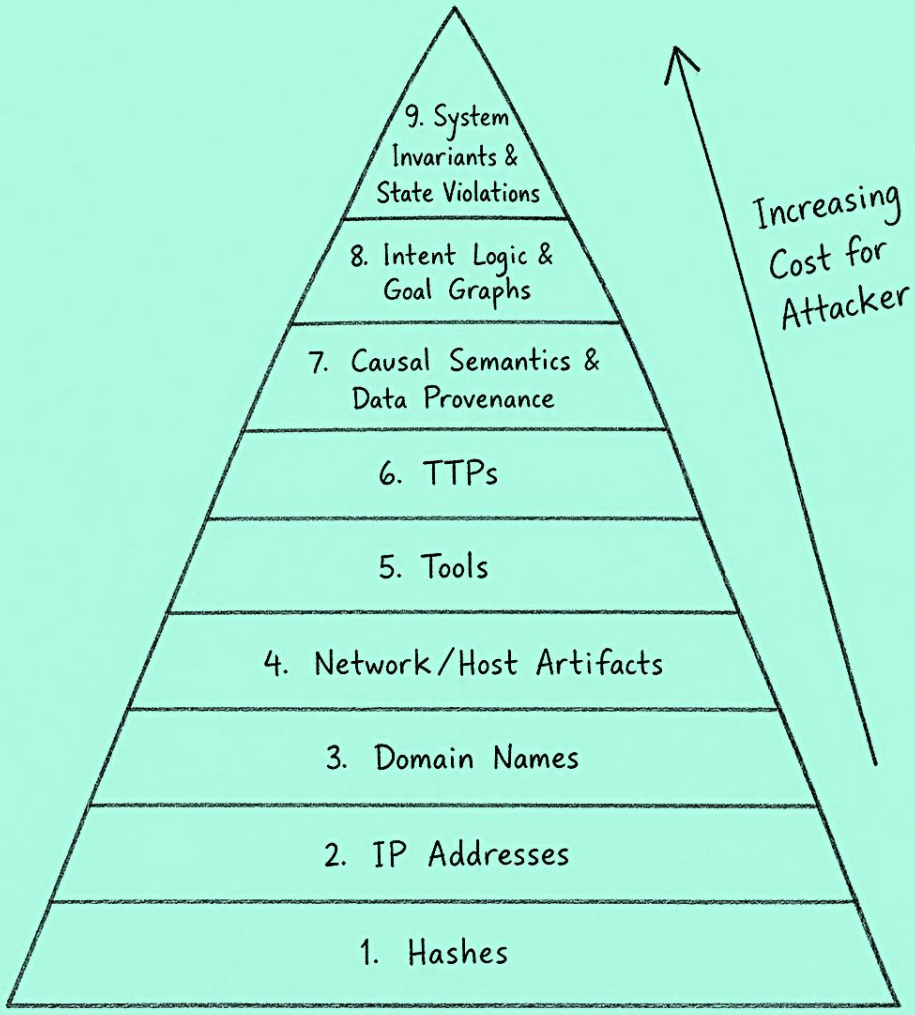
•**پیامد:** یک تکنیک مانند T1055 (Process Injection) ده‌ها پیاده‌سازی گوناگون (Process Hollowing, APC Queue, Shim Injection, AtomBombing و...) دارد. نوشتن دیتکشن برای یکی از آن‌ها کل خانه را در ماتریس سبز می‌کند، در حالی که مهاجم از زاویه ندیده‌شده دیگری از همان تکنیک عبور می‌کند.

۳ .ناتوانی در حذف نویز علّی مشکل:

ابزارهای پیاده‌کننده TTP، زمان را معیار پیوند رویدادها قرار می‌دهند، نه جریان داده.

•**پیامد:** ابزار امنیتی یک تکنیک کشف اعتبارنامه و یک تکنیک ارتباط شبکه‌ای را در فاصله ۵ دقیقه می‌بیند و آن‌ها را به عنوان حمله ثبت می‌کند، حتی اگر این دو رویداد هیچ ربط ارگانیکی به هم نداشته باشند (تولید خطامثبت زیاد)، یا برعکس، چون در پنجره زمانی متفاوتی رخ داده‌اند پیوند آن‌ها را گم می‌کند.

هرم درد گسترش یافته



تمرین اول– بخش امتیازی

- با نصب یک ماشین مجازی آسیب پذیر به بلوایترنال (مثلا ویندوز هفت وصله نشده)، با استفادهه از کدهای موجود برای بهره کشی از این آسیب پذیری، سعی کنید به سیستم نفوذ کنید.
- از همه مراحل نصب و حمله و نتایج گزارش مصور تهیه کنید.
- مهلت حداکثر ده روز از امروز

تمرین اول – بخش اجباری

- استاکس نت ویروسی بود که در زمان دولت اوباما و با همکاری مشترک آمریکا، اسراییل و ناتو بعنوان یکی از نخستین سلاح های سایبری در سال ۱۳۸۹ با حمله به نطنز قصد نابودی تاسیسات اتمی ایران را داشت.
- با مطالعه منابع معتبر درباره این رخداد، یک گزارش تحلیلی مستقل و مصور تهیه کنید که ساختار آن بر پایه دو چارچوب مکمل استوار باشد
- :۱. تحلیل زنجیره حمله
- ۲. تحلیل رفتاری و نگاشت به TTPها
- گزارش نباید صرفاً ترجمه یک مستند موجود باشد و باید شامل دیاگرام‌های تحلیلی، جداول مقایسه‌ای و ارجاع به منابع معتبر باشد).مهلت ارسال: شنبه آینده